Pet-Care Connecting System

Complete Student Guide & System Workflow

A comprehensive guide explaining the architecture, features, and end-to-end data flow of the modern full-stack Pet-Care web application.

---

1. System Architecture Overview

The Pet-Care Connecting System is a robust **Full-Stack Application** built with modern technologies, divided into three main layers:

🖥️ Frontend (Client Side)

The user interface where pet owners, vets, and admins interact.

React 19 TypeScript

React Router Axios

Leaflet Maps Recharts

Runs on Node.js (Port 3000)

⚙️ Backend (Server Side)

The core engine handling business logic, APIs, and ML requests.

Spring Boot 3.4 Java 17

Spring Security JWT

RESTful APIs

Runs on Apache Tomcat (Port 8080)

🗄️ Database & External APIs

Where data is stored securely and intelligent AI services.

MySQL 8.0 Hibernate ORM

Gemini AI API Groq API

MySQL Port 3306

2. How the Authentication Works (JWT Flow)

Security is handled via JSON Web Tokens (JWT). The system has three user roles: `USER` (Pet Owner), `VET` (Veterinarian), and `ADMIN` .

> → **1. User Logs In:** Frontend sends email and password to `/api/auth/login` via Axios.
>
> → **2. Backend Validates:** Spring Security & `AuthController` check credentials against the MySQL database.
>
> → **3. Token Generation:** If correct, `JwtService` generates an encrypted JWT string containing the user's ID and Role.
>
> → **4. Token Storage:** React receives the token and stores it in `localStorage` and the `AuthContext` .
>
> → **5. Protected APIs:** For every future request (e.g., adding a pet), Axios attaches the token to the `Authorization: Bearer [TOKEN]` header.
>
> → **6. Backend Filtering:** `JwtAuthFilter` intercepts the request, verifies the token signature, and grants access.

3. Core Features & Module Workflows

🐶 Pet & Health Management

Goal: Allow users to register their pets, track their health, and list medications.

- **Frontend:** `PetsPage.tsx` and `MedicationsPage.tsx` provide a dashboard to add/edit pets.
- **Backend:** `PetController` & `MedicationController` manage requests. `PetService` maps to the `pets` database table via JPA repository.
- **Relationship:** One user can have multiple pets (`@OneToMany`). One pet can have multiple medications.

📅 Appointments & Vet Consultations

Goal: Connect pet owners with veterinarians for physical appointments or digital consultations.

- **Booking (Owner):** User visits `AppointmentsPage.tsx`, selects a vet from the dropdown, sets a date, and submits the request.
- **Processing (Backend):** `AppointmentController` saving it as "PENDING" in MySQL.
- **Approval (Vet):** A Vet logs in, sees requests assigned to them, and changes status to "CONFIRMED" or "CANCELLED".
- **Consultations:** Follows a similar flow but for text/audio advice (`ConsultationPage.tsx`).

🤖 AI Chatbot & ML Diagnostics

Goal: Provide instant AI-driven advice for pet health concerns.

> → **User Input:** User types a symptom (e.g., "My dog is limping") in `ChatbotPage.tsx` .
>
> → **API Call:** React sends text to backend `/api/chatbot/ask` .
>
> → **External AI Service:** Spring Boot's `ChatAssistantService` formats the prompt (acting as an expert vet) and sends it to the **Google Gemini API** (or Groq).
>
> → **Response:** The AI model evaluates the symptoms and returns actionable, safe veterinary advice to the UI.

🏥 Surgeries & Adoptions

- **Surgical Records:** Vets can log surgical procedures on pets using `SurgicalPage.tsx`. This acts as a permanent medical history.
- **Adoptions:** Users looking to rehome pets can use `AdoptionPage.tsx`. It creates a listing in the `adoption_listings` table, visible to everyone.

🗺️ Nearby Vets Mapping

Uses **react-leaflet** to display an interactive map on the `NearbyVetsPage.tsx` frontend. It maps registered veterinarians, allowing users to find clinics close to their physical location graphically.

🔔 Notifications System

Goal: Keep users informed about appointments and system updates.

- **Sending:** Vets and Admins use `NotificationsPage.tsx` to send targeted or global messages.
- **Storage:** Saved in the `notifications` table connected to specific User IDs.
- **Receiving:** The frontend polls or fetches unread notifications via `NotificationController` to display in the header bell icon.

4. Database Schema Dictionary

A high-level view of the MySQL relational tables:

Table Name	Purpose / Contents
`users`	Stores ID, Email, encrypted Password, and Role (USER, VET, ADMIN).
`pets`	Belongs to a user. Dog/Cat species, breed, age, and medical history.
`medications`	Linked to a pet. Name, dosage, and frequency instructions.
`appointments`	Links a Pet (Owner) and a Vet. Has Date, Time, Reason, and Status.
`consultations`	Digital advice logs between users and vets. Includes vet notes.

Table Name	Purpose / Contents
`surgical_procedures`	Logged by vets. Includes surgery type, outcome, and date.
`adoption_listings`	Pets available for adoption. Includes descriptions and contact info.
`notifications`	Messages sent from system/vets to users. Tracks "isRead" status.

5. Quick Setup & Execution Guide

Step 1: Database Setup

1. Open MySQL Workbench / XAMPP.
2. Create a database: `CREATE DATABASE petcare_db;`
3. Ensure MySQL is running on port 3306 with root/root credentials (or change in `application.properties`).

Step 2: Run Backend (Spring Boot)

1. Open `pet-care-backend` in IntelliJ IDEA / Eclipse.
2. Wait for Maven to download dependencies.
3. Run `BackendApplication.java`. The server starts on `http://localhost:8080`.

(Hibernate will automatically create all tables in MySQL)

Step 3: Run Frontend (React)

1. Open terminal in `pet-care-frontend`.
2. Run `npm install`.
3. Run `npm start`.
4. Application opens in browser at `http://localhost:3000`.

Note for Students: Follow the data flow starting from the React Components, through the Axios API calls, hitting the Spring Boot Controllers, passing business logic to Services, and finally persisting in the MySQL Repositories.